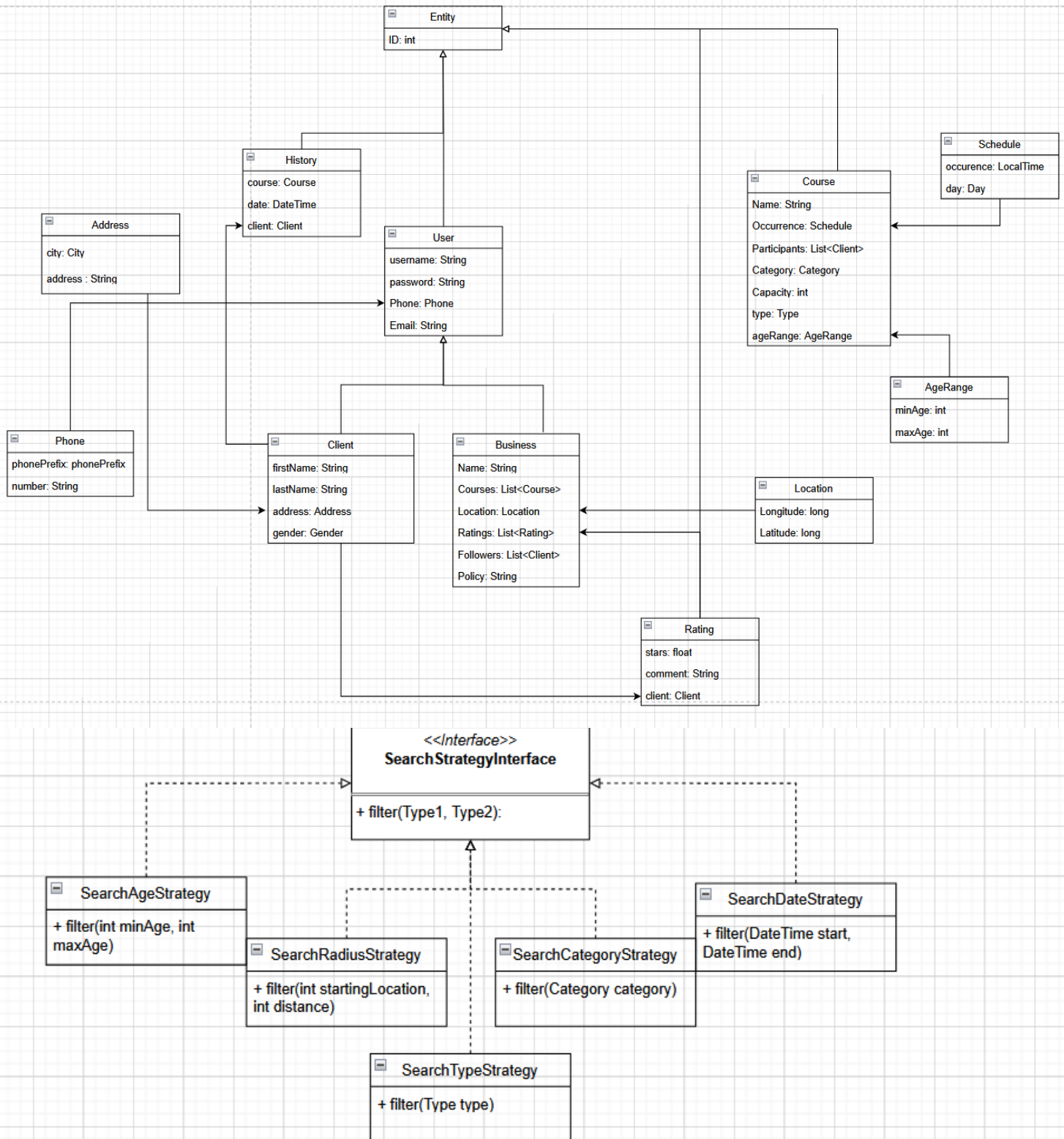
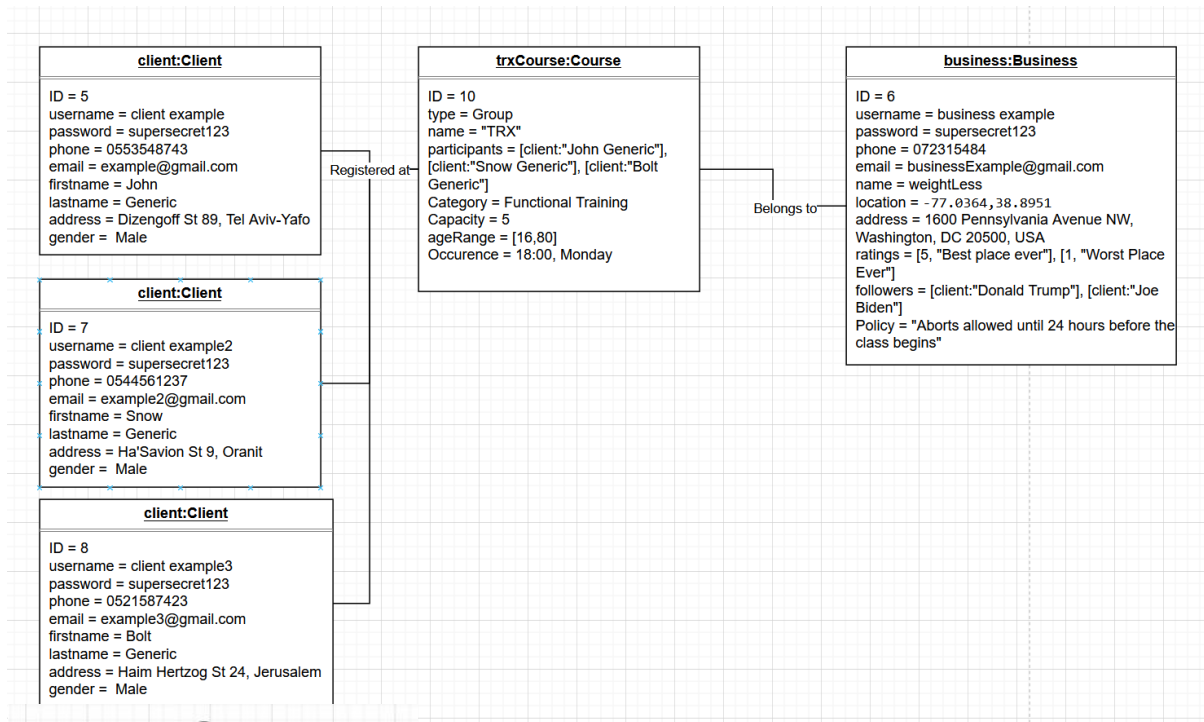


מסמר ניתוח – Workout

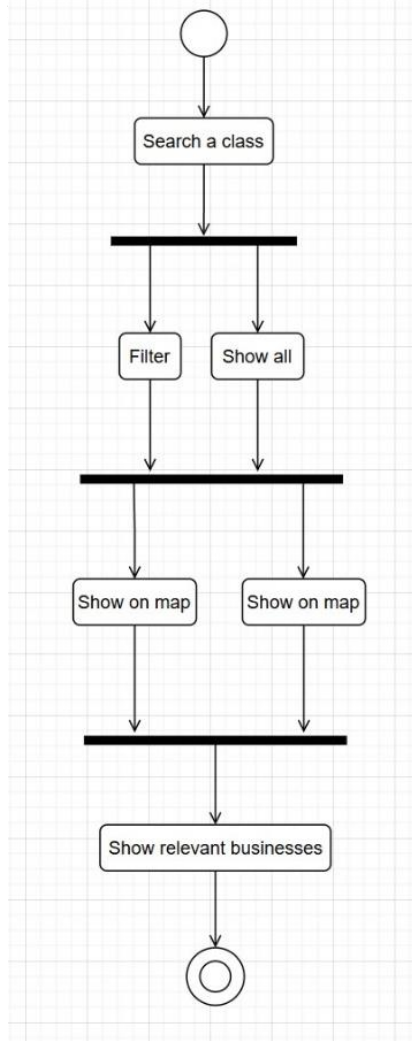
- Class Diagram 1



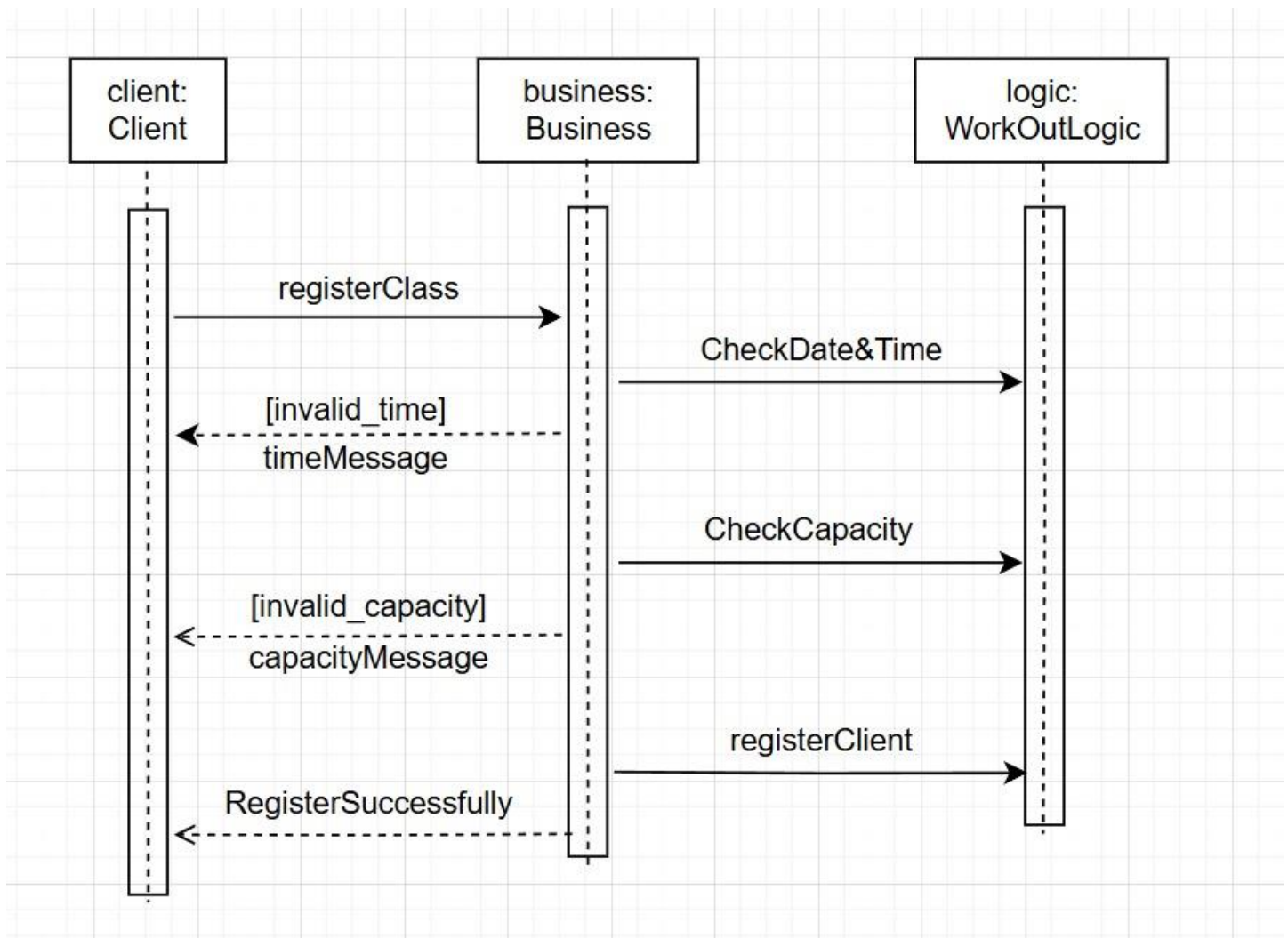
- Object Diagram .2



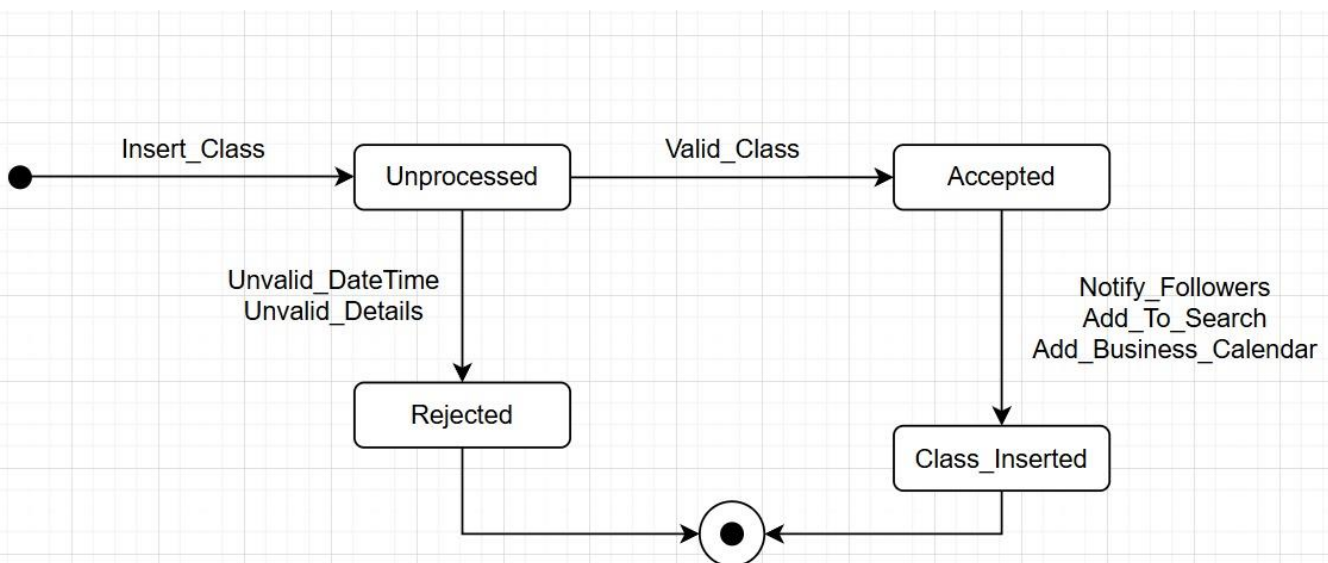
- Activity Diagram .3



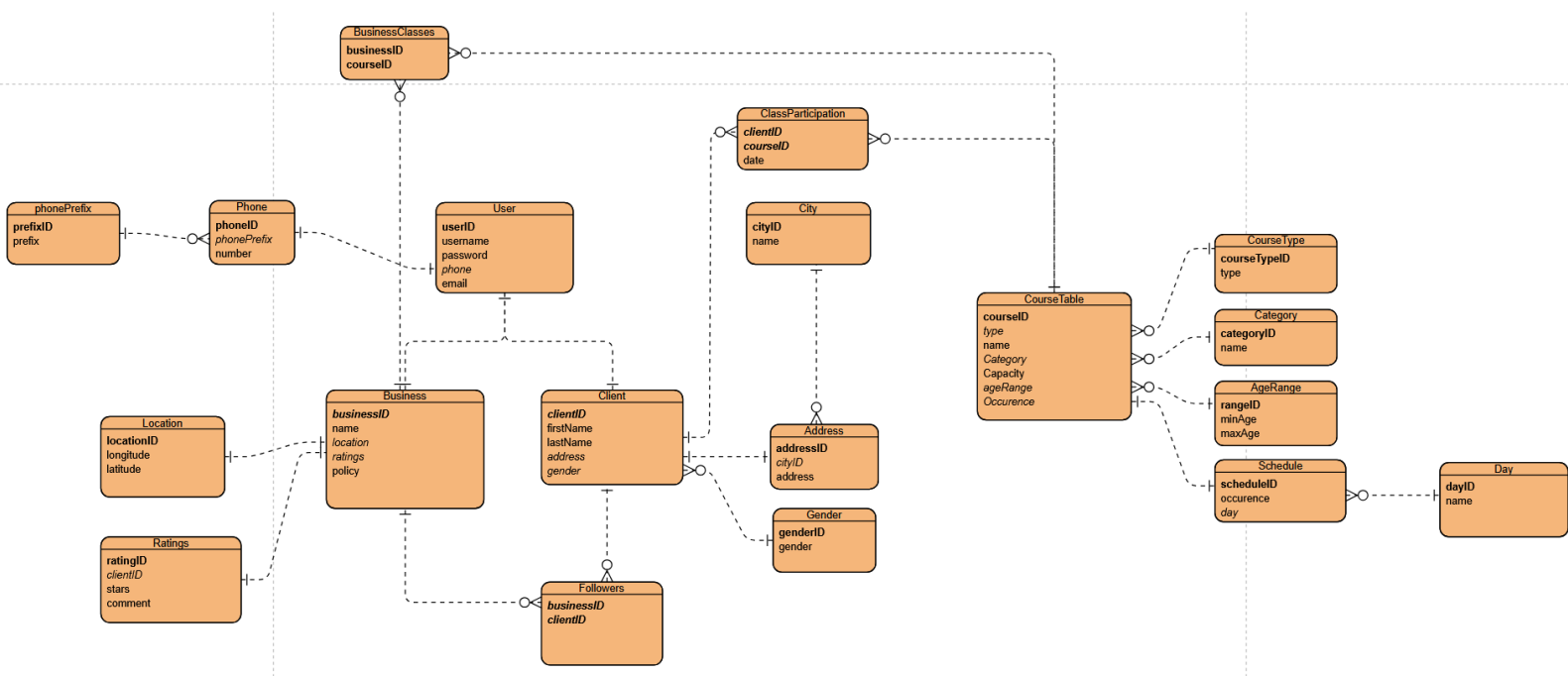
– Sequence Diagram .4



– State Machine Diagram .5



– ERD .6



System Screens Structure

מבנה המסכים במערכת

רשימת המסכים עבור המתאמן:

- חיבור\הרשמה: התחברות או הרשמה למערכת.
- עמוד הבית: מכיל יומן ומעבר לשאר הדפים הקיימים.
- חיפוש חוג: מתאמן יוכל לחפש חוג ולהרשם אליו.
- היסטוריה: מסך המכיל את כל השיעורים אליהם המתאמן נרשם.
- דירוג: מתאמן יוכל לדרג את בתי עסק בהם התאמן.
- שינוי פרטים: המתאמן יוכל לשנות את פרטיו.
- מפה: כל בתי העסק יופיעו ללא סינון ברדיוס שנקבע מראש.
- יומן: כל השיעורים אליו נרשם המתאמן יופיעו ביומן מסודר.
- הודעות: מסך בו יופיעו כל ההודעות וההתראות שהמתאמן קיבל.

רשימת המסכים עבור בית העסק:

- חיבור\הרשמה: התחברות או הרשמה למערכת.
- עמוד הבית: מכיל יומן ומעבר לשאר הדפים הקיימים.
- שינוי פרטים: מסך בו בית העסק יוכל לשנות את פרטיו.
- הוספת שיעור: בית העסק יוכל להוסיף שיעור חדש.
- עדכון שיעור: בית העסק יוכל לעדכן שיעור קיים.
- יומן: מסך המכיל את כל השיעורים ששבית העסק מציע.
- דירוגים ועוקבים: מסך המכיל את כל הדירוגים והעוקבים של בית העסק.

2. מעבר בין מסכים:

תחילה, המשתמש נכנס למסך ההתחברות לאפליקציה. ממסך זה המשתמש יכול להירשם במידה והוא לא משתמש רשום או להתחבר עם שם המשתמש והסיסמה על מנת להגיע לדף הבית.

עבור משתמש שהוא מתאמן:

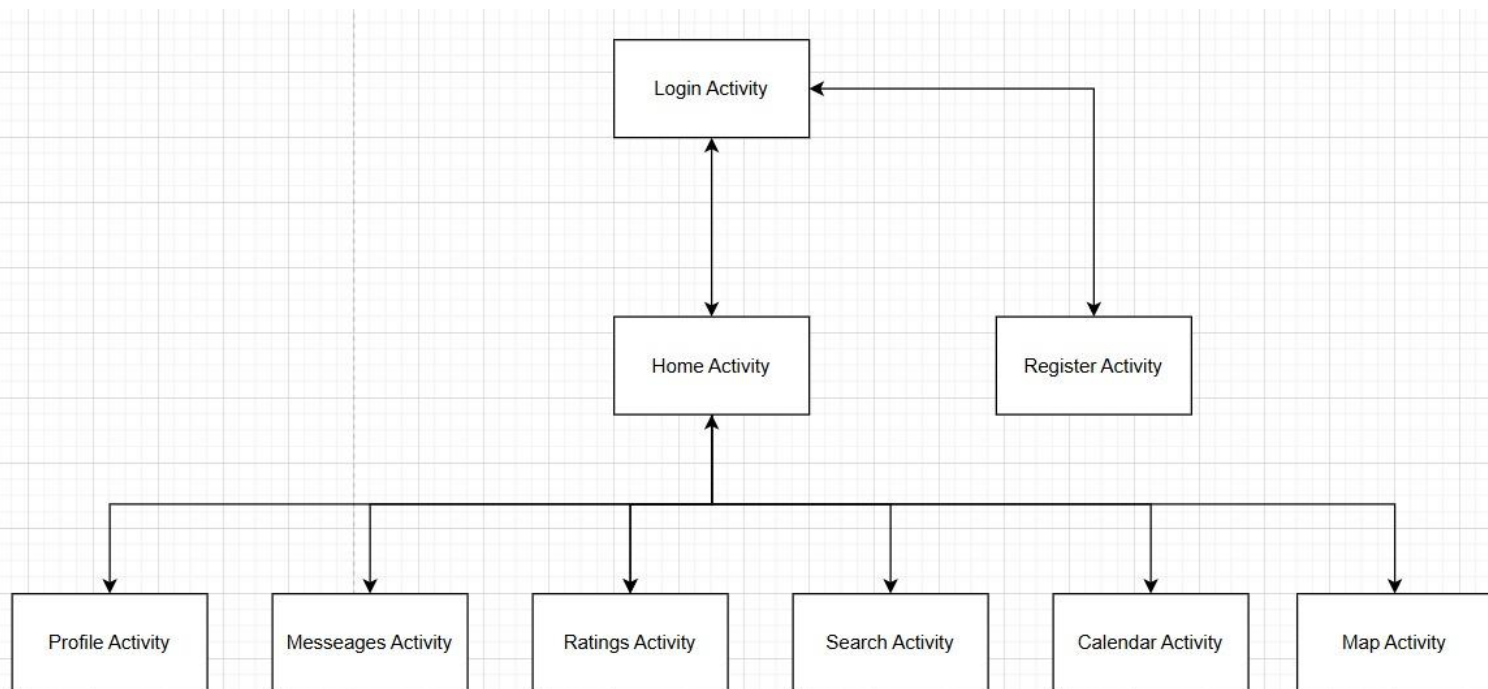
ממסך הבית, המשתמש יכול להגיע למסכים: הודעות אחרונות, היסטוריית אימונים, יומן אימונים, חיפוש חוג עדכון הפרטים שלו, מפה ודירוג בתי עסק.

מכל הדפים (פרט למסך הבית) ניתן להגיע חזרה למסך הבית.

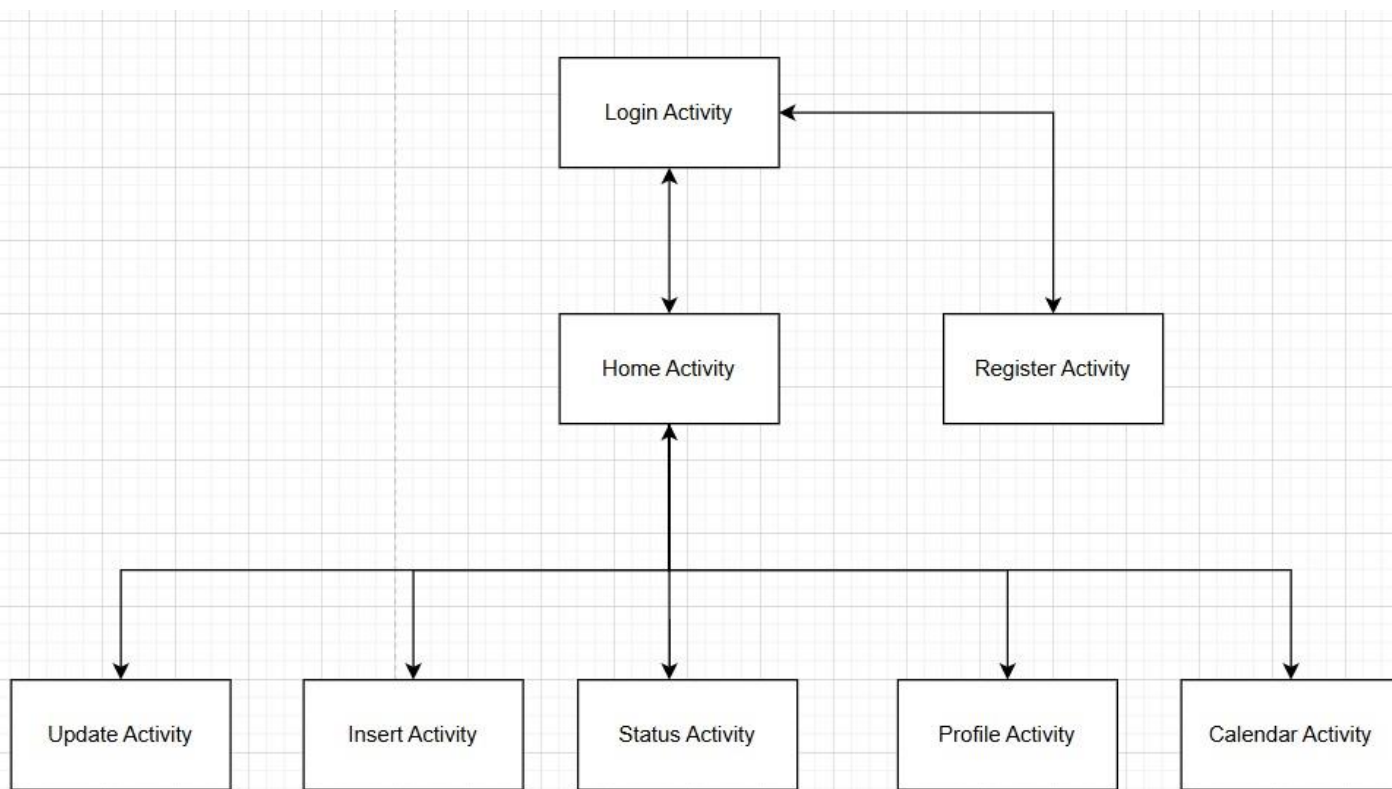
עבור משתמש שהוא בית עסק:

ממסך הבית, המשתמש יוכל להגיע למסכים: הוספת שיעור, שינוי פרטים, עדכון שיעור, יומן ודירוגים ועוקבים.

3. תרשים זרימת מסכי לקוח:



תרשים זרימת מסכי עסק:



מאפייני המטלה – תבניות עיצוב:

בפרוייקט זה אנחנו נשתמש בארכיטקטורת MVVM. לארכיטקטורה זו יש המון יתרונות (שלדעתנו עולים על החסרונות) וזו השיטה המקובלת והמודרנית כיום לפיתוח תוכנה. הפרוייקט שלנו אינו גדול במיוחד ולכן MVVM – תתאים לנו בצורה מיטבית והיעילה ביותר.

יתרונותיה המרכזיים:

1. הפרדה מוחלטת בין התצוגה (View and ViewModel) לבין הלוגיקה (Model).
2. כל חלק מתמקד בתפקיד שלו.
 - Model: אחראי על הלוגיקה והמימוש בפרוייקט.
 - ViewModel: שכבה המקשרת בין ה-Model ל-View.
 - View: ממשק המשתמש ומה שהאפליקציה מציגה בפועל.
3. הפרדה זו מאפשרת בצורה מיטבית את בדיקת פונקציונאליות המערכת, שיפורה ותיקונה במידת הצורך.

חסרונותיה המרכזיים:

1. עבור פרוייקטים גדולים, עיצוב שכבת ה-ViewModel עלול להיות מאתגר.
2. Debug בשכבת ה-View עלול להיות קשה מכיוון שה-Data bindings הם ייצוגיים בלבד (במקום לעשות debug עם break points).

בעקבות החלוקה בארכיטקטורה זו, ניתן לתחזק ולשפר את האפליקציה בצורה יעילה. אם נרצה להוסיף כפתור בדף כלשהו, להוסיף פונקציה מסויימת או לשנות מסך – תמיד נדע לאיזו שכבה אנו אמורים לגשת. כשלכל אחת יש את התפקיד המרכזי שלה – חלוקת תוכן האפליקציה מתבצעת באופן הגיוני וטבעי, דבר המקל על המפתחים בעת הפיתוח, הבדיקה והתחזוקה השוטפת. בנוסף, לא מתבזבזים משאבים "מיותרים" על איתור בעיות הדורשות טיפול או מעבר על קוד כלשהו באפליקציה.